Etapa 2: Logica pe date (JavaScript)

Tehnologii Web · Proiect de semestru · Etapa 2

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Inginerie Industrială și Robotică
Disciplina: Tehnologii Web, proiect de semestru
Anul universitar: 2026–2027
Coordonatori de proiect: Emilia PĂSĂRIN, Alexandru RIZEA, Andrei-Alexandru STAICU

Enunțul etapei

Să se mute datele aplicației din pagina HTML într-un fișier JavaScript, sub forma unui array de obiecte, și să se scrie funcțiile de bază ale aplicației: listarea titlurilor, numărarea elementelor active, căutarea, adăugarea cu validare, comutarea stării și ștergerea. Funcțiile se testează în consola browserului, iar rezultatele se afișează grupate pe secțiuni.

Fișierul JavaScript se leagă de pagina din Etapa 1, dar **nu modifică pagina**: nu folosește obiectul `document`, nu tratează evenimente și nu schimbă nimic din ce se vede pe ecran. Astfel, logica aplicației se dezvoltă și se verifică separat de interfață. În această etapă nu se folosesc npm și Node.js; acestea apar în Etapa 3.

Etapa introduce cel mai important principiu pentru React: **funcțiile nu modifică datele primite, ci întorc o listă nouă**. Funcțiile scrise acum vor fi reutilizate aproape neschimbate în aplicația React, în Etapa 5.

La finalul etapei veți putea:

- reprezenta datele unei aplicații printr-un array de obiecte cu identificator unic;
- folosi metodele `map`, `filter`, `find` și `reduce`;
- scrie funcții care nu modifică datele primite (funcții imutabile);
- valida datele înainte de a le adăuga în listă;
- citi și interpreta mesajele din consola browserului.

Noțiuni introductive

Array de obiecte și identificatorul unic

Fiecare element al listei este un obiect cu aceleași câmpuri ca în README. În plus, fiecare element primește un **id**: un număr întreg care nu se repetă. Id-ul va fi cheia elementelor în listele React (Etapa 4) și cheia primară în baza de date (Etapa 10).

```
const taskuri = [
  { id: 1, titlu: "Tema la Baze de date", gata: false, prioritate: "mare" },
  { id: 2, titlu: "Citit capitolul 3",    gata: true,  prioritate: "medie" },
  { id: 3, titlu: "Cumpărat cafea",       gata: false, prioritate: "mica" },
];
```

Metodele de array

Funcțiile din această etapă se scriu cu patru metode ale obiectului `Array` (tabelul 2.1). Niciuna dintre ele nu modifică array-ul pe care este apelată.

Tabelul 2.1: Metodele de array folosite în etapă

Metodă	Ce întoarce	Exemplu	Utilizare
`map`	un array nou, cu fiecare element transformat	`taskuri.map((t) => t.titlu)`	lista titlurilor, comutarea stării
`filter`	un array nou, cu elementele care îndeplinesc o condiție	`taskuri.filter((t) => !t.gata)`	elementele active, căutarea, ștergerea
`find`	primul element care îndeplinește condiția sau `undefined`	`taskuri.find((t) => t.id === 2)`	căutarea după id
`reduce`	o singură valoare, calculată parcurgând lista	vezi Pasul 6	id-ul maxim

Funcțiile de tip săgeată, precum `(t) => t.titlu`, întorc automat expresia din dreapta săgeții. Dacă se folosesc acolade, `(t) => { ... }`, valoarea trebuie întoarsă explicit cu `return`.

Imutabilitatea

O funcție imutabilă nu modifică datele primite, ci construiește și întoarce date noi. În practică se folosesc două forme, ambele bazate pe operatorul *spread* (`...`):

- pentru un array: în loc de `lista.push(x)` se scrie `[...lista, x]`;
- pentru un obiect: în loc de `t.gata = !t.gata` se scrie `{ ...t, gata: !t.gata }`.



Fig. 2.1: Diferența dintre modificarea pe loc și crearea unui array nou

Motivul practic apare în Etapa 5: React decide dacă trebuie să redeseneze interfața comparând referințele datelor. Un array modificat pe loc are aceeași referință, deci React consideră că nu s-a schimbat nimic, iar interfața nu se actualizează.

Consola browserului

Rezultatele se afișează cu `console.log(...)` și se citesc în fila **Console** din DevTools, deschisă cu **F12** (sau direct cu **Ctrl + Shift + J**). În dreapta fiecărui mesaj apar fișierul și linia din care a fost scris, ceea ce ajută la identificarea erorilor.

Adaptarea la tema proprie

Denumirile fișierului, ale array-ului și ale funcțiilor se adaptează temei proprii, conform tabelului 2.2. Ultima coloană se completează înainte de începerea implementării. Dacă tema are câmpuri suplimentare, căutarea le include, iar validarea le acoperă (de exemplu un an de apariție între 1450 și anul curent sau o durată pozitivă).

Tabelul 2.2: Denumirile pe tema proprie

Element	TaskFlow	BookShelf	WatchList	Tema proprie
fișierul	taskuri.js	carti.js	filme.js	
array-ul	taskuri	carti	filme	
valorile fixe	PRIORITATI	FORMATE	TIPURI	
numărarea	numaraActive	numaraNecitite	numaraDeVazut	
comutarea stării	comutaGata	comutaCitita	comutaVazut	
căutarea	după titlu	după titlu sau autor	după titlu	

Identificatorii din cod (variabile, funcții, fișiere) se scriu fără diacritice; mesajele afișate în consolă se scriu cu diacritice.

Desfășurarea lucrării

Pasul 1: Crearea fișierului JavaScript și legarea lui de pagină

Lângă `index.html` se creează fișierul JavaScript (pentru TaskFlow, `taskuri.js`). În `index.html` se adaugă o singură linie, chiar înainte de `</body>`:

```
  <script src="taskuri.js"></script>
</body>
```

Se deschide pagina în browser, apoi consola (**Ctrl + Shift + J**). Pentru verificare, se poate scrie temporar în fișier `console.log("merge");` și se reîncarcă pagina (**F5**). Dacă numele sau calea fișierului sunt greșite, consola afișează o eroare, iar mesajele proprii lipsesc (figura 2.2).

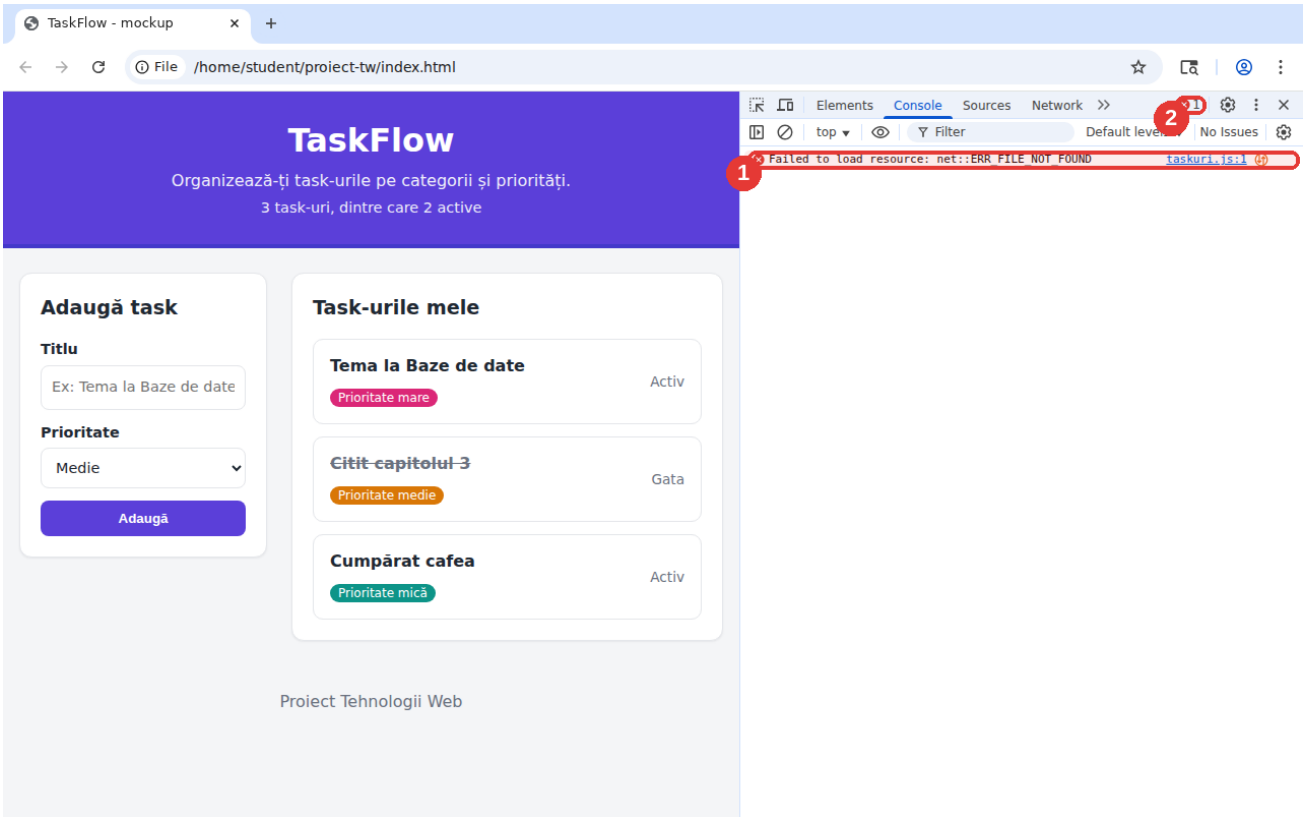


Fig. 2.2: Eroarea afișată când fișierul JavaScript nu este găsit

1. Mesajul `Failed to load resource: net::ERR_FILE_NOT_FOUND`: browserul nu a găsit fișierul indicat în `<script src="...">`. Se verifică numele fișierului și faptul că se află în același folder cu `index.html`.

2. Numărul de erori din pagină. Un cerc roșu cu o valoare diferită de zero indică o problemă care trebuie rezolvată înainte de a continua.

Pasul 2: Datele de test și valorile permise

În fișier se scriu cele trei elemente de test din README, fiecare cu un `id`, precum și constanta cu valorile permise pentru eticheta fixă:

```
const taskuri = [ /* cele trei elemente din README, fiecare cu id */ ];

const PRIORITATI = ["mica", "medie", "mare"];
```

Pasul 3: Listarea titlurilor

Funcția primește lista și întoarce un array cu titlurile. Este singura funcție de listare dată complet, ca model pentru celelalte:

```
function listeazaTitluri(lista) {
  return lista.map((t) => t.titlu);
}

console.log("Titluri:", listeazaTitluri(taskuri).join(", "));
```

Pasul 4: Numărarea elementelor active

Funcția `numaraActive(lista)` întoarce numărul elementelor nebifate: se filtrează elementele cu starea `false`, apoi se citește proprietatea `length` a rezultatului.

Pasul 5: Căutarea după titlu

Funcția `cautaDupaTitlu(lista, text)` întoarce elementele al căror titlu conține textul căutat, indiferent de literele mari sau mici. Ambele texte se convertesc cu `toLowerCase()`, iar comparația se face cu `includes()`. Astfel, căutarea „tema” găsește și „Tema la Baze de date”.

Pasul 6: Adăugarea unui element, cu validare

Funcția `adaugaTask(lista, titlu, prioritate)` are trei responsabilități:

1. **validarea**: titlul curățat cu `trim()` nu poate fi gol, iar eticheta trebuie să existe în lista de valori permise; în caz contrar, se afișează un mesaj în consolă și se întoarce lista neschimbată;
2. **calculul id-ului**: noul id este cel mai mare id existent plus unu;
3. **întoarcerea unui array nou**, cu elementul adăugat la final.

Calculul id-ului se face cu o funcție separată, bazată pe `reduce`:

```
function nextId(lista) {
  return lista.reduce((max, t) => Math.max(max, t.id), 0) + 1;
}
```

Varianta `lista.length + 1` nu este corectă: după ștergerea unui element produce id-uri duplicate. De exemplu, dintr-o listă cu id-urile 1, 2, 3 se șterge elementul 2; lungimea devine 2, iar noul element ar primi id-ul 3, care există deja. Structura funcției de adăugare:

```
function adaugaTask(lista, titlu, prioritate = "medie") {
  const titluCurat = titlu.trim();
  // 1. dacă titluCurat este gol: mesaj în consolă și return lista
  // 2. dacă prioritatea nu este în PRIORITATI: mesaj și return lista
  // 3. se construiește obiectul nou, cu id-ul dat de nextId(lista)
  // 4. return [...lista, nou]
}
```

Pasul 7: Comutarea stării și ștergerea

Ambele funcții primesc lista și un id și întorc o listă nouă:

- `comutaGata(lista, id)` folosește `map`: elementul cu id-ul căutat se înlocuiește cu o copie cu starea inversată, `{ ...t, gata: !t.gata }`, iar celelalte elemente rămân neschimbate;
- `stergeTask(lista, id)` folosește `filter` și păstrează toate elementele cu alt id.

Pasul 8: Testele din consolă

La finalul fișierului se adaugă testele manuale, grupate pe secțiuni. Ultimul rând din secțiunea *Adăugare* dovedește imutabilitatea: după adăugare, array-ul inițial trebuie să aibă în continuare trei elemente.

```
console.log("--- Citire ---");
console.log("Titluri:", listeazaTitluri(taskuri).join(", "));
console.log("Active:", numaraActive(taskuri));
console.log("Căutare 'tema':", listeazaTitluri(cautaDupaTitlu(taskuri,
"tema")).join(", "));

console.log("--- Adăugare ---");
let lista = adaugaTask(taskuri, "Proiect Tehnologii Web", "mare");
console.log("Lista nouă:", lista.length, "task-uri");
console.log("Originalul a rămas cu:", taskuri.length, "task-uri");

console.log("--- Modificare și ștergere ---");
lista = comutaGata(lista, 1);
console.log("După bifarea id 1, active:", numaraActive(lista));
lista = stergeTask(lista, 3);
console.log("După ștergerea id 3:", listeazaTitluri(lista).join(", "));

console.log("--- Validare ---");
adaugaTask(lista, "   ");
adaugaTask(lista, "Ceva", "urgenta");
```

După salvarea fișierului și reîncărcarea paginii, consola afișează rezultatele din figura 2.3.

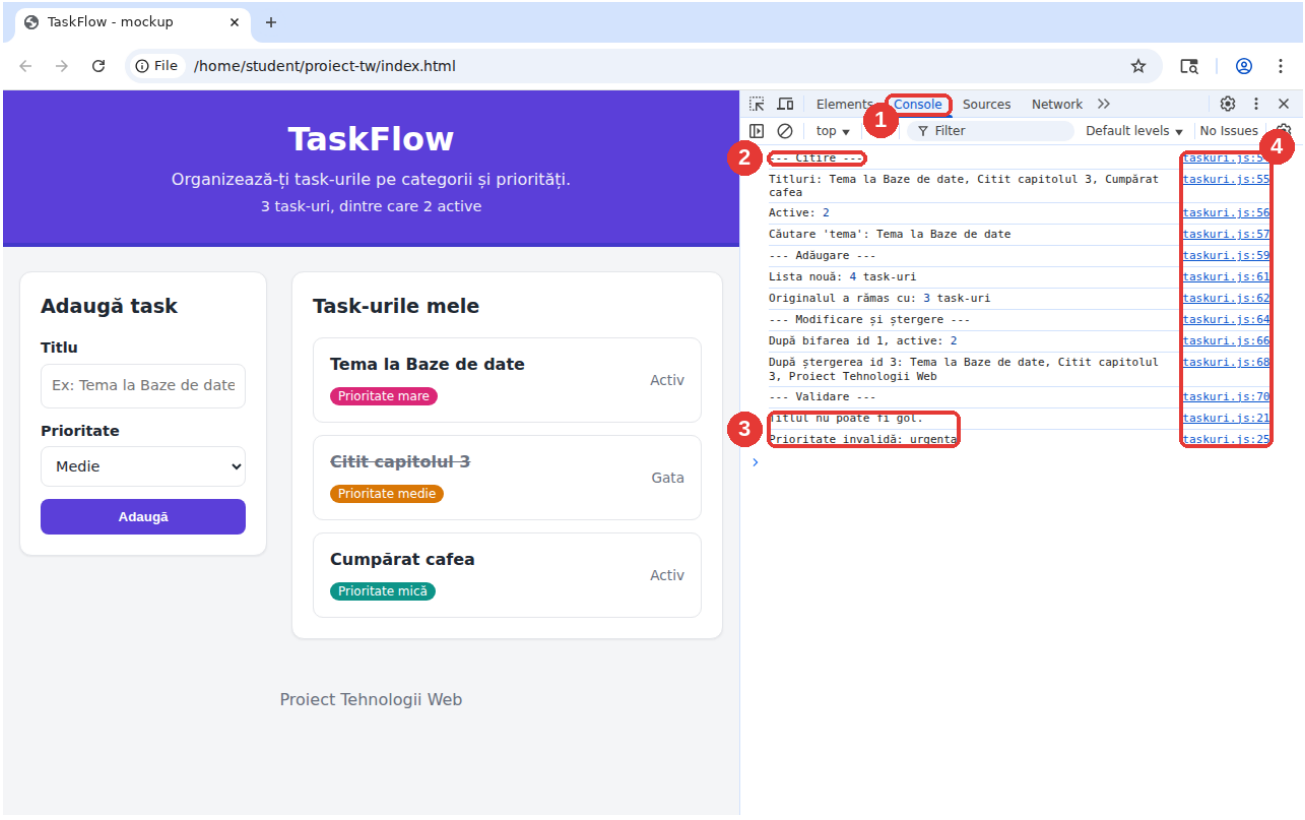


Fig. 2.3: Rezultatele testelor în consola browserului (TaskFlow)

1. Fila **Console** din DevTools.
2. Titlurile de secțiune fac rezultatul ușor de citit.

3. Mesajele produse de validare: titlul gol și eticheta inexistentă sunt respinse, fără ca programul să se oprească.
4. Fișierul și linia din care provine fiecare mesaj. Un clic pe link deschide codul în fila *Sources*.

Pagina din Etapa 1 rămâne vizual neschimbată: fișierul JavaScript nu modifică interfața.

Pasul 9: README, jurnalul AI și publicarea pe GitHub

În README se adaugă secțiunea etapei și se actualizează stadiul:

```
## Stage 2: data logic
Plain JavaScript, no DOM. `taskuri.js` holds the array and the functions
that read and change it. Results are printed in the browser console (F12).

## Status
- [x] Stage 1: static mockup
- [x] Stage 2: data logic in JavaScript
□ Stage 3: Vite and React project
```

Se completează `ai-log/etapa-02.md`, după același șablon ca în Etapa 1, și se actualizează, dacă este cazul, secțiunea *AI usage* din README. Apoi:

```
git add .
git commit -m "Stage 2: data logic in JavaScript"
git push
```

Tabelul de verificare din README

După publicarea codului, în README se adaugă tabelul de verificare al etapei, după modelul din Etapa 1: coloana *Where* conține permalink-ul către liniile de cod corespunzătoare, iar coloana *How to check* indică modul de verificare. Tabelul se publică într-un commit separat, `Stage 2: checklist`.

Tabelul 2.3: Tabelul de verificare al Etapei 2

ID	Requirement	Where (permalink)	How to check
S2-R1	JS file linked, logs on page load	index.html#L.. (script)	open page, F12
S2-R2	3+ items with id, name, state, tag	taskuri.js#L..-L..	read
S2-R3	list, count, search, add, toggle, delete	taskuri.js#L..-L..	console output
S2-R4	add rejects empty name and invalid tag	taskuri.js#L..-L..	last 2 console lines
S2-R5	original array unchanged after add	taskuri.js#L..	console line
S2-R6	README Stage 2 section + AI log	README.md, ai-log/etapa-02.md	read
S2-R7	commit “Stage 2” pushed	link to the commit	commit history

Lista de verificare a etapei

Codurile de mai jos sunt aceleași ca în tabelul de verificare din README.

☐ **S2-R1** Fișierul JavaScript este legat de pagină și scrie în consolă la deschiderea acesteia.

☐ **S2-R2** Array-ul conține cel puțin trei elemente, fiecare cu id, nume, stare și etichetă fixă.

☐ **S2-R3** Funcțiile de listare, numărare, căutare, adăugare, comutare și ștergere funcționează corect.

☐ **S2-R4** Adăugarea respinge numele gol și eticheta inexistentă, cu mesaj în consolă.

☐ **S2-R5** Array-ul inițial rămâne neschimbat după adăugare, fapt demonstrat în consolă.

☐ **S2-R6** README-ul conține secțiunea *Stage 2*, iar `ai-log/etapa-02.md` este completat.

☐ **S2-R7** Commit-ul `Stage 2: ...` este publicat pe GitHub.

Criterii de calitate urmărite: id-ul nou se calculează ca maxim existent plus unu; niciuna dintre funcții nu modifică array-ul primit; testele sunt grupate pe secțiuni, iar rezultatul se poate citi ușor.

Probleme frecvente

Tabelul 2.4: Probleme frecvente și rezolvarea lor

Simptom	Cauză probabilă	Rezolvare
array-ul original se modifică	`push` sau atribuire directă pe obiect	`[...lista, nou]` și `{ ...t, camp: valoare }`
căutarea „tema” nu găsește „Tema”	comparația ține cont de literele mari	`toLowerCase()` pe ambele texte
id-uri duplicate după ștergere	id calculat din `lista.length`	maximul existent plus unu (`reduce`)
funcția întoarce `undefined`	funcție săgeată cu acolade, fără `return`	`(t) => t.titlu` sau `return` explicit
consola rămâne goală sau afișează `ERR_FILE_NOT_FOUND`	eticheta `<script>` lipsește sau are cale greșită	verificarea căii; eticheta înainte de `</body>` (figura 2.2)
consola afișează rezultate vechi	pagina este servită din cache	reîncărcare forțată cu Ctrl + Shift + R

Cerințe până la etapa următoare

Până la etapa următoare, fișierul cu funcțiile, README-ul actualizat (cu tabelul de verificare) și jurnalul `ai-log/etapa-02.md` trebuie să fie publicate pe GitHub. În Etapa 3 se inițializează proiectul React cu Vite, iar datele și funcțiile din această etapă sunt integrate în aplicație. Pentru Etapa 3 este necesar Node.js, versiunea 20 sau mai nouă, instalat de pe nodejs.org (varianta LTS).